Task Behavior Graphs: A Domain-Driven Model for Profiling Queue Worker Tasks under Uncertain Resource Demand

Anonymous Author(s)

Abstract

TODO

Keywords: queue workers; task profiling; workload characterization; resource demand; worker occupancy; behavior graphs; observability; instrumentation; admission control; overload regulation.

1 Introduction

1.1 Queue Workers as Task Execution Systems

Queue-based execution is a common design pattern for decoupling work production from work execution. A producer submits a task instance to a queue, and one or more workers later dequeue, reserve, execute, retry, acknowledge, or discard that work. In small systems, this mechanism is often treated as a background-job abstraction. In production systems, however, queue workers frequently become a general execution layer for heterogeneous and operationally significant work: data replication, cache rebuilds, database repair, media transformation, external service calls, batch imports, validation, notification delivery, maintenance jobs, and other forms of asynchronous processing.

This distinction matters because a queue worker is not merely a consumer of messages. It is an execution boundary. Once a task attempt enters a worker, it may consume CPU time, memory, disk I/O, network bandwidth, runtime allocations, dependency capacity, database connections, leases, locks, semaphores, retry budgets, and tenant-level concurrency slots. Some of these costs appear as conventional hardware or runtime resource usage. Others are better understood as worker-occupancy costs: the attempt holds a worker slot, a lease, a connection, or another bounded execution resource for some duration. A task attempt that consumes little CPU but occupies a worker slot for a long time may reduce effective throughput and increase queueing delay more severely than a shorter compute-dominant attempt.

The operational unit that profiling must explain is therefore not only the worker process, container, or machine. It is the task attempt executed inside a queue-worker boundary, together with the task kind, task-instance features, and execution context that shaped that attempt. Section 2 defines these terms more precisely. Informally, a task kind describes what work is requested, a task instance is the concrete queued unit of that kind, and a task attempt is one concrete execution of that instance.

However, the measurements easiest to collect are often not task-attempt-level measurements. Operating systems and container runtimes expose CPU, memory, I/O, and network counters for processes, cgroups, containers, or nodes. Worker frameworks expose aggregate queue depth, worker utilization, task duration, failure counts, and retry counts. Observability systems expose metrics, logs, traces, and runtime profiles. These observations may show that a worker is busy, that a queue is growing, or that a dependency is slow, but they do not by themselves explain which task kind, task instance, or task attempt created the pressure, which features were relevant, or which context factors amplified the observed behavior.

This paper therefore treats queue workers as task execution systems rather than passive queue consumers. Under this framing, task profiling becomes a structured estimation problem: given a task kind, task-instance features, and execution-context observations, what resource demand, worker occupancy, uncertainty, and operational risk should the system expect for a task attempt, and what profiling policy is justified? Different task kinds may require different observations, which makes a single universal metric set insufficient. The difficulty is deeper, however: even attempts of the same task kind may produce substantially different costs under different execution contexts.

1.2 Task Cost Is Conditional, Not Intrinsic

A natural first approximation is to assign a stable cost to a task kind, a queue, or a payload-size bucket. Under this approximation, a `replicate-partition` task with a one-gigabyte payload, an image transformation task with a fixed input size, or a validation task for a small document should have a predictable resource footprint. Resource-aware cluster managers already reason about workloads through resource requests, constraints, scheduling decisions, and observed usage [12]. However, prior work also shows that static or user-provided resource assumptions may be insufficient for dynamic workloads. Quasar, for example, motivates inferring resource requirements from application behavior and performance constraints rather than relying only on user-specified reservations [5]. For queue-worker profiling, this implies that task kind and declared size can provide a useful baseline, but they should not be treated as the complete cost model.

Consider two task instances of the same replication task kind with the same payload size, the same declared operation, and the same nominal priority. The first instance is attempted against a healthy replica set with stable network connectivity, low peer latency, and small replication lag. It completes in roughly one minute. The second instance is attempted against a degraded replica set with unstable network connectivity, peer delay, retransmissions, and repeated retry or backoff intervals. It occupies a worker slot for fifty minutes. The two instances are similar by task kind and declared task features, but their attempts are not similar by operational behavior.

The difference is not explained by the intrinsic task description alone. The declared intrinsic work may appear similar: read a range of data, transfer or apply it to a peer, verify progress, and commit a replication checkpoint. The observed behavior, however, is shaped by execution context. Network instability can increase transfer time and retries. Peer degradation can increase dependency wait time. Replication lag can correlate with a larger reconciliation backlog. Disk pressure can delay reads or writes. Worker pressure can increase local scheduling delay. The same baseline task kind can therefore produce different resource demand and different worker occupancy under different contexts. This view is consistent with production workload studies that characterize task usage shapes and observe heterogeneous, dynamic behavior in large compute clusters [1, 10, 13].

This conditional behavior is not specific to replication. A transformation task kind may exhibit compute-dominant behavior when record count and algorithm mode dominate CPU time, but memory-pressure-sensitive behavior when allocation volume or working-set size dominates. A task kind that calls external services may be shaped by dependency latency, timeout policy, fan-out width, connection-pool availability, and retry behavior. A database repair or cache rebuild task kind may depend on hidden state such as segment fragmentation, compaction debt, replica divergence, dirty data volume, or disk queue depth. Prior workload-classification systems such as Paragon and Quasar also treat workload behavior as something that must be classified with respect to heterogeneity, interference, or performance constraints rather than inferred from a name alone [4, 5].

Task profiling should therefore not assign a single intrinsic cost to a task kind. A useful model must treat task cost as conditional: intrinsic task features define a baseline, execution context can amplify or suppress that baseline, and residual uncertainty remains when observed

behavior is not explained by known features. This residual uncertainty matters because rare slow executions can dominate system-level behavior in large-scale distributed systems [2]. In other words, the profiling problem is not to attach a universal cost label to a task kind, but to estimate task-kind-specific behavior under a given execution context. Before selecting metrics or instrumentation, the system must identify which task-instance features and context factors are relevant for the task kind being profiled.

1.3 Observations Are Not a Profiling Model

Metrics, logs, traces, counters, and runtime profiles are necessary inputs to task profiling, but they are not a profiling model by themselves. Distributed tracing systems such as Dapper demonstrate the value of tracing for understanding large-scale distributed behavior, and modern telemetry systems provide structured metrics, traces, and profiles for production systems [9, 11]. These mechanisms expose observations about execution. They do not, however, define which observations are relevant for a particular task kind, which observations should be interpreted as context factors, root-cause candidates, symptoms, or outcomes, or how observations should be converted into estimates of resource demand and worker occupancy.

The relevance problem appears immediately when different task kinds are compared. A replication task kind may require observations about network condition, peer health, replication lag, retries, and worker-slot time. A transformation task kind may require input size, record count, CPU time, allocation behavior, and memory pressure. A task kind that calls external services may require dependency latency, timeout rate, retry count, and connection-pool wait time. A simple validation task kind may require only minimal accounting. A flat observation set shared by all task kinds either misses important task-specific factors or collects unnecessary signals on the hot path.

Raw observations also do not define influence structure. For example, network degradation may increase transfer time, dependency latency, retries, wall-clock duration, and worker-slot occupancy. Treating network throughput, latency, retry count, and wall-clock duration as independent cost factors can count the same underlying degradation multiple times. Conversely, treating only the final wall-clock duration as the cost hides the mechanism that produced it. A profiling model therefore needs to distinguish context factors, root-cause candidates, symptoms, outcomes, derived signals, and residuals.

A further gap is feature mapping. A raw observation such as packet loss, replica lag, connection-pool wait time, or allocation rate is not yet a model input with stable meaning. It must be interpreted, normalized, and mapped into a canonical feature such as network instability, replica health, dependency capacity pressure, or runtime allocation pressure. Without this layer, two domains may use different raw metrics for the same underlying influence, and two similar metric names may represent different operational meanings. A task profiling model must therefore specify how raw domain-specific observations become canonical features used for estimation.

Finally, observations have a cost. Tracing, metric collection, high-cardinality labels, runtime profiling, and telemetry export can add CPU, memory, allocation, storage, and network overhead. Dapper was explicitly designed around low overhead and application-level transparency, and OpenTelemetry documentation similarly emphasizes that collecting more telemetry can increase overhead and that high-cardinality metric tags can increase memory usage [8, 11]. Sampling mechanisms, including tail sampling, are therefore profiling-policy choices rather than properties of observations themselves [7]. A profiling system must decide what to observe, at what level of detail, for which task kind or attempt, and under which overhead budget.

For these reasons, the problem addressed in this paper is not simply how to collect more telemetry. The problem is how to interpret observations as part of a task-kind-specific profiling model. The next subsection introduces influence domains as the vocabulary for assigning

semantic roles to observations and Task Behavior Graphs as the structure for connecting those roles within a task behavior model.

1.4 From Influence Domains to Task Behavior Graphs

The preceding discussion suggests that task profiling requires more than collecting additional observations. A profiling system must know what an observation means for a given task kind, whether it describes the task identity, intrinsic demand, execution context, resource demand, worker occupancy, outcome, uncertainty, or risk, and whether it should affect resource estimation, worker-occupancy estimation, uncertainty, or profiling policy. We use *influence domains* as the semantic vocabulary for making these distinctions.

An influence domain groups model elements by their role in task behavior. Intrinsic-demand features describe the work contained in the task instance, such as payload size, record count, operation mode, or partition size. Execution-context features describe the environment in which a task attempt executes, such as network instability, dependency health, replica state, cache warmth, disk pressure, or worker pressure. Resource-demand dimensions describe active resource use, such as CPU time, memory peak, disk I/O, and network I/O. Worker-occupancy dimensions describe bounded execution capacity held by the attempt, such as worker-slot time, lease time, connection hold time, lock hold time, or tenant-level concurrency time. Outcome, uncertainty, and risk domains describe how the attempt completed, how predictable the estimate is, and what consequences follow from estimation or control error.

Influence domains provide the vocabulary, but they do not by themselves define a model. The model must also describe how domain-specific features influence one another for a particular task kind or task family. This paper introduces *Task Behavior Graphs* for that purpose. A Task Behavior Graph represents a task kind or task family as a typed graph whose nodes belong to influence domains and whose edges encode modeled relationships such as baseline demand, contextual amplification, additive overhead, derived symptoms, outcome links, uncertainty indicators, and risk propagation. The graph is not assumed to prove causality by itself; rather, it makes the model's influence assumptions explicit so that they can be estimated, validated, calibrated, or revised.

For example, in a replication task kind, payload size may contribute to network and disk demand, network instability may amplify retry behavior and worker-slot time, and prolonged worker-slot time may increase queue-starvation risk. In a transformation task kind, record count and algorithm mode may instead dominate CPU time, allocation behavior, and memory pressure, while network instability may be irrelevant or only weakly relevant. The graph therefore does not encode a universal metric list. It encodes which features and relationships are relevant for the task kind or family being modeled.

The graph structure is associated with a task kind or task family rather than constructed independently for every task attempt. The task kind or family provides the structure: relevant features, resource and occupancy dimensions, influence roles, estimated sensitivities, and confidence in those relationships. A task instance supplies intrinsic feature values, such as payload size or record count. A task attempt supplies execution-context observations, attempt state, worker state, and runtime observations. Under the model, graph evaluation produces task-kind-specific estimates of resource demand, worker occupancy, uncertainty, residuals, and operational risk. A separate profiling-policy selector can then use these estimates to choose an observation strategy, such as minimal accounting, detailed accounting, sampled tracing, tail-aware profiling, diagnostic profiling, or isolated diagnostic execution.

This framing also avoids the assumption that a profiling system can maintain a globally complete metric list. Different domains may expose different raw metrics for the same underlying influence, and different task kinds may require different subsets of those influences. Instead of requiring every metric to be known by the core model, Task Behavior Graphs operate over canonical features. Domain-specific metrics can be mapped into these features through task

behavior manifests and domain packs, while unexplained behavior remains visible as residual error and reduced confidence. The result is a task-kind-specific profiling model: raw observations are interpreted through influence domains, connected through a behavior graph, and used to select the amount and kind of profiling justified for the task attempt.

1.5 Contributions and Paper Organization

This paper argues that queue-worker task profiling should be treated as a task-kind-specific behavior modeling problem rather than as a problem of collecting more raw telemetry. The central claim is that task attempts should be profiled through a model that separates task identity, intrinsic demand, execution context, resource demand, worker occupancy, uncertainty, and operational risk. Within this framing, the paper makes the following contributions.

First, we define a task cost model for queue workers that separates intrinsic demand, contextual amplification, additive overhead, worker occupancy, and residual uncertainty. This model treats task cost as conditional behavior rather than as a fixed scalar attached to a task kind, queue, or payload-size bucket.

Second, we introduce influence domains as a semantic decomposition of queue-worker task behavior. These domains distinguish task identity, intrinsic demand, execution context, resource demand, worker occupancy, outcomes, uncertainty, risk, and profiling observations. The purpose of this decomposition is to avoid flat metric lists, separate context factors from symptoms and outcomes, and make explicit which kinds of observations are relevant for a task kind.

Third, we propose Task Behavior Graphs as a task-kind- or task-family-level representation that connects canonical features across influence domains. A Task Behavior Graph describes how intrinsic task features and execution-context factors contribute to resource demand, worker occupancy, uncertainty, and operational risk. The graph provides the structure, while task instances and attempts provide concrete feature values, context observations, outcomes, and residuals for evaluation.

Fourth, we describe a metrics-to-features model based on feature registries, metric descriptors, task behavior manifests, and domain packs. This mechanism allows domain-specific raw observations to be mapped into canonical features without requiring a globally complete metric list in the core profiling model. It also makes missing or unexplained influences visible through residual error and reduced confidence.

Fifth, we outline a profiling-policy selection flow that uses predictability, context sensitivity, worker occupancy, operational consequences, and profiling overhead to choose an appropriate observation strategy. Depending on the task kind, attempt context, and risk profile, the selected strategy may range from minimal accounting to detailed accounting, sampled tracing, tail-aware profiling, diagnostic profiling, or isolated diagnostic execution.

The remainder of the paper is organized as follows. Section 2 defines the terminology and notation used throughout the paper. Sections 3 and 4 introduce the background and formulate the profiling problem. Sections 5 through 8 define the cost model, influence domains, task taxonomy, and Task Behavior Graphs. Sections 9 through 13 describe observation mapping, profiling methods, worker observation points, uncertainty, risk, and profiling-policy selection. Section 14 introduces task behavior manifests and domain packs, and Section 15 applies the model to representative task kinds. The remaining sections discuss evaluation methodology, limitations, validity threats, related work, and conclusions.

2 Core Terminology and Notation

This section defines only the core terms required to state the problem and introduce the model. The paper uses a larger terminology set, but the complete reference glossary is deferred to

Appendix A so that the main text can proceed from the problem statement to the model without front-loading implementation and lifecycle details.

The central distinction is that a task kind describes what work is requested, whereas task behavior describes how that work executes under a particular execution context. Raw observations provide evidence, canonical features feed the model, influence domains assign semantic roles, and Task Behavior Graphs connect features across domains to estimate demand, occupancy, uncertainty, and risk.

2.1 Core Terms

Table 1: Core terms used in the main model.

Term	Definition
task	A generic unit of work handled by a queue-worker system. The term is used only when the distinction between task kind, task instance, and task attempt is not important.
task kind	The semantic type of work submitted to a queue. A task kind describes what work is requested, but it does not fully determine resource demand, worker occupancy, or risk under every execution context.
task instance	A concrete logical unit of work placed into a queue. A task instance belongs to a task kind and may produce one or more execution attempts.
task attempt	One concrete execution of a task instance. Retries, lease expiration, worker failure, cancellation, or rescheduling may cause a single task instance to have multiple attempts. Resource usage, worker occupancy, observations, outcomes, and residuals are usually measured or estimated at attempt granularity.
worker	An executor that dequeues, reserves, executes, retries, acknowledges, discards, or fails task attempts. In this paper, a worker is treated as an execution boundary rather than merely a message consumer.
task behavior	The observed or estimated execution pattern of a task attempt or task kind under a given execution context, including resource demand, worker occupancy, outcome, uncertainty, and risk-relevant effects.
execution context	The environment and state under which a task attempt executes. Execution context includes worker state, node pressure, dependency health, network condition, disk pressure, cache state, tenant pressure, retry state, and other runtime or domain-specific conditions.
observation	Any measurement, event, trace, profile sample, counter, log entry, or domain-specific signal exposed by a queue, worker runtime, operating system, container runtime, dependency, tracing system, or domain component.

Term	Definition
canonical feature	A typed and normalized model input derived from task metadata, execution context, raw observations, or signals. Task Behavior Graphs operate over canonical features rather than arbitrary raw metrics.
influence domain	A semantic role assigned to a feature, observation, output, or model concept. Influence domains separate task identity, intrinsic demand, execution context, resource demand, worker occupancy, outcome, uncertainty, risk, and profiling observations.
resource demand	The expected or required active resource need of a task attempt along a resource dimension, such as CPU time, memory peak, disk I/O, network I/O, runtime allocation, or accelerator usage. Resource demand is what the model estimates before or during execution.
resource usage	The observed actual use of a resource during execution. Resource usage is measured after or during execution and is distinct from estimated demand.
worker occupancy	Bounded execution capacity held by a task attempt over time, such as worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, or tenant-level concurrency time.
worker-slot time	The time during which a task attempt holds a worker execution slot. This is the core worker-occupancy dimension in queue-worker systems.
uncertainty	Unreliability or unpredictability of a task behavior estimate. Uncertainty may come from missing features, hidden state, measurement error, drift, heavy tails, multimodality, or insufficient history.
operational risk	The consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Operational risk is modeled separately from task cost.
Task Behavior Graph	A typed graph associated with a task kind or task family. It connects canonical features across influence domains and supports estimation of resource demand, worker occupancy, uncertainty, and operational risk. A Task Behavior Graph makes modeled influence assumptions explicit; it is not assumed to prove causality by itself.
profiling policy	The selected observation and instrumentation strategy for a task kind, task attempt, or execution context. A profiling policy may choose minimal accounting, detailed accounting, tracing, sampling, tail-aware profiling, diagnostic profiling, or isolated diagnostic execution.

2.2 Notation

Table 2: Notation used throughout the paper.

Symbol	Meaning
$\mathcal{K}$	The set of task kinds.
$\mathcal{I}$	The set of task instances.
$\mathcal{A}$	The set of task attempts.
$i \in \mathcal{I}$	A task instance.
$a \in \mathcal{A}$	A task attempt.
$k(i) \in \mathcal{K}$	The task kind of task instance i.
$i(a) \in \mathcal{I}$	The task instance executed by attempt a.
$\mathcal{D}$	The set of influence domains.
$\mathcal{F}$	The set of canonical features.
z_a	The canonical feature vector available for attempt a after feature mapping and normalization.
$\mathcal{R}$	The set of resource-demand dimensions.
$\mathcal{O}$	The set of worker-occupancy dimensions.
$D_r(a)$	Observed resource demand or usage for resource dimension $r \in \mathcal{R}$ during attempt a.
$\widehat{D}_r(a)$	Estimated resource demand for resource dimension $r \in \mathcal{R}$ during attempt a.
$U_o(a)$	Observed worker occupancy for occupancy dimension $o \in \mathcal{O}$ during attempt a.
$\widehat{U}_o(a)$	Estimated worker occupancy for occupancy dimension $o \in \mathcal{O}$ during attempt a.
$\epsilon_r(a)$	Residual error for resource-demand dimension r, defined as the part of observed resource behavior not explained by the estimate.
$\epsilon_o(a)$	Residual error for worker-occupancy dimension o, defined as the part of observed occupancy behavior not explained by the estimate.
$\mathrm{Risk}(a)$	The operational-risk estimate for attempt a.
$\mathcal{G}_k$	The Task Behavior Graph associated with task kind k.
V_k	The set of nodes in Task Behavior Graph $\mathcal{G}_k$.
E_k	The set of edges in Task Behavior Graph $\mathcal{G}_k$.
$v \in V_k$	A node in Task Behavior Graph $\mathcal{G}_k$.
$e \in E_k$	An edge in Task Behavior Graph $\mathcal{G}_k$.
$\delta(v)$	The influence domain assigned to graph node v.

Symbol	Meaning
w_e	The estimated sensitivity or weight associated with graph edge e.
γ_e	The confidence value associated with graph edge e.

3 Background and Motivation

This section provides the technical background needed to formulate the queue-worker task-profiling problem. It does not introduce the proposed model in full. Instead, it motivates why task profiling cannot be reduced to process-level resource monitoring, queue-level metrics, or generic tracing. The section first characterizes queue workers as task-attempt execution systems, then reviews the resource and workload properties that make task behavior conditional and variable. It finally explains why observations require semantic interpretation and why profiling must be constrained by overhead.

3.1 Queue-Worker Execution and Task Attempts

Queue-based execution separates work submission from work execution. A producer submits a task instance to a queue, and a worker later dequeues or reserves that instance, starts an execution attempt, performs the required work, and eventually acknowledges, retries, cancels, fails, or discards it. This lifecycle appears simple when the queue is treated as a background-job mechanism. It becomes more complex when the queue is the execution layer for heterogeneous work such as replication, repair, cache rebuilds, imports, media processing, external API fan-out, notifications, validation, or maintenance.

The relevant unit for profiling is the task attempt. A task kind describes the semantic type of work, and a task instance describes one concrete queued unit of that kind. However, retries, lease expiration, worker failure, timeout, cancellation, or rescheduling may cause a single task instance to produce multiple attempts. Each attempt can observe a different worker state, node state, dependency state, retry state, or domain state. As a result, resource usage, worker occupancy, outcome, and residual error are usually most precise at attempt granularity rather than at task-kind or queue granularity.

This distinction matters because a worker is an execution boundary, not only a message consumer. Once an attempt starts, it may consume active resources such as CPU time, memory, disk I/O, network I/O, runtime allocations, or accelerator resources. It may also hold bounded execution capacity, such as a worker slot, a lease, a lock, a database connection, a semaphore permit, a dependency quota, or a tenant-level concurrency slot. These held capacities are not always visible as high CPU or memory usage, but they can still reduce effective throughput and increase queueing delay.

Therefore, task profiling in queue-worker systems must account for both active resource demand and worker occupancy. A short compute-dominant attempt and a long dependency-bound attempt may impose different kinds of cost. The former may consume CPU intensely for a short time. The latter may consume little CPU but hold a worker slot, a connection, or a lease for a long duration. Treating both attempts only through process-level utilization hides this difference.

3.2 Resource Demand as a Multidimensional Quantity

Distributed resource-management systems commonly reason about multiple resource dimensions rather than a single scalar load. Cluster managers and resource allocation mechanisms consider

dimensions such as CPU, memory, storage, disk, and other capacity constraints when placing or admitting work [6, 12]. This background is important for queue-worker profiling because a task attempt cannot be represented faithfully by one undifferentiated cost value.

For queue-worker tasks, the resource vector must include active-resource dimensions and may also need dimensions that are specific to execution capacity. Active resource demand includes CPU time, memory peak, allocation volume, disk reads and writes, network transfer volume, and accelerator usage when relevant. Worker-occupancy dimensions include worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, and tenant concurrency time. The two groups are related, but they are not identical. A task can be CPU-heavy with short occupancy, wait-heavy with long occupancy, or mixed across several phases.

The distinction between demand and pressure is equally important. Task demand is an estimate or observation about the attempt itself. Resource pressure is a condition of the execution environment: CPU saturation, memory pressure, disk queue depth, network instability, dependency saturation, or worker-pool utilization. A CPU-intensive task and a CPU-saturated node are different model objects. The first describes work performed by the attempt; the second describes the context in which the attempt executes. Confusing demand with pressure makes it difficult to decide whether an attempt is intrinsically expensive, contextually amplified, or merely observed under degraded conditions.

A useful profiling model must therefore preserve resource dimensions, occupancy dimensions, and context dimensions separately before any policy-specific score is computed. A scalar score may be useful for admission, ranking, or alerting, but it is a projection of a richer model. It should not replace the underlying resource and occupancy estimates.

3.3 Workload Variability and Context Dependence

Production workloads are heterogeneous and dynamic. Prior workload studies show that task usage, cluster heterogeneity, and workload behavior vary substantially across time, machines, and task populations [1, 10, 13]. Systems such as Paragon and Quasar also motivate classification and inference of workload behavior rather than relying only on static resource assumptions or user-provided reservations [4, 5]. For this paper, the relevant consequence is that task kind, queue name, or payload size can provide a useful baseline, but they do not fully determine an attempt's behavior.

Consider a replication task kind. Two task instances may have the same declared operation and similar payload size. The first attempt may run against a healthy replica set with stable network conditions, low peer latency, and low replication lag. The second may run against a degraded replica set with unstable network conditions, retransmissions, peer delay, retry backoff, or disk pressure. The intrinsic work may appear similar, but the second attempt can hold a worker slot much longer and may produce greater retry or queueing effects. The difference is not explained by task identity alone; it depends on execution context.

The same pattern appears in other task families. A transformation task may be mostly shaped by record count, algorithm mode, CPU pressure, allocation behavior, and memory pressure. A task that calls external APIs may be shaped by dependency health, fan-out width, timeout policy, connection-pool availability, and retry state. A database repair, compaction, cache rebuild, or migration task may be shaped by hidden domain state such as fragmentation, dirty data volume, replication divergence, cache warmth, or disk queue depth. In each case, the model must identify which features are relevant for that task kind rather than apply one global feature list.

Variability is also important at the tail. Large-scale distributed systems can be strongly affected by rare slow operations, stragglers, and high-percentile behavior [2,3]. For queue workers, a rare attempt that holds a worker slot, lock, lease, or dependency connection for an unusually long time can reduce effective capacity even if average behavior looks acceptable. A profiling

model that only tracks mean duration or average resource usage can therefore underestimate operational risk.

This motivates a conditional view of task behavior. Intrinsic task features provide a baseline for expected work. Execution context can amplify, suppress, or otherwise reshape that baseline. Remaining mismatch appears as residual error, uncertainty, drift, or unmodeled behavior. The purpose of profiling is not only to record what happened, but to improve this conditional estimate for future attempts and to identify when the estimate is unreliable.

3.4 Observability Signals and Profiling Overhead

Metrics, logs, traces, runtime profiles, and domain events provide evidence about task execution, worker state, dependencies, and the surrounding system. Tracing systems such as Dapper and modern telemetry systems make it possible to observe distributed execution paths, spans, metrics, and runtime behavior [9, 11]. These observations are necessary for task profiling, but they are not sufficient as a profiling model.

Raw observations do not carry stable model semantics by themselves. A metric may be a counter, gauge, histogram, runtime profile, or domain-specific event. It may represent a task feature, an execution-context factor, an active-resource observation, a worker-occupancy observation, a symptom, an outcome, or a risk indicator. For example, packet loss can be interpreted as part of network instability; retry count may be a symptom of dependency failure, network instability, timeout policy, or idempotency failure; wall-clock duration may be an outcome that combines CPU work, I/O, dependency wait, retry backoff, and local scheduling delay. Without semantic roles, a flat list of observations can lead to double counting or to estimates that hide the mechanism that produced the observed behavior.

Feature mapping is therefore a separate step from observation collection. Raw metrics and events must be converted into typed, normalized, canonical features before they can be used by a task behavior model. Different domains may expose different raw observations for similar underlying influences, while similar metric names may have different meanings across systems. A model that operates directly over arbitrary raw metrics is difficult to reuse across task kinds and domains. A model that operates over canonical features can separate domain integration from graph evaluation.

Observation also has cost. Instrumentation, tracing, runtime profiling, high-cardinality metric labels, sampling decisions, and telemetry export can add CPU overhead, allocations, memory pressure, storage volume, and network traffic. Dapper was designed with low overhead as a central concern, and OpenTelemetry similarly treats telemetry volume, sampling, and performance overhead as operational concerns [7, 8, 11]. For queue-worker profiling, this means that more observation is not always a better policy. The profiling level must be selected with respect to task kind, uncertainty, consequence, and overhead budget.

3.5 Motivating Gap

The background above identifies a gap between available observations and the model needed for queue-worker task profiling. Resource-management systems reason about multiple resources, workload studies show that task behavior is dynamic and heterogeneous, and observability systems expose metrics, traces, logs, and profiles. However, these pieces do not by themselves define how a queue-worker system should attribute behavior to logical task attempts, determine which features are relevant for a task kind, distinguish intrinsic demand from execution context, account for worker occupancy, or select a profiling policy under overhead constraints.

The gap is most visible when tasks are compositional or context-sensitive. A single task kind may include CPU processing, network transfer, external dependency calls, retries, commits, and cleanup. Some quantities compose by summation, others by maximum, critical path, retry expansion, or occupancy-specific rules. Some observations are root or context factors,

while others are derived symptoms or outcomes. Without an explicit model of these roles and relationships, a profiling system cannot reliably determine how to combine observations or when additional instrumentation is justified.

The next section formulates this gap as a profiling problem. The problem is to estimate task-attempt behavior from task identity, intrinsic features, execution-context observations, and available profiling signals, while keeping resource demand, worker occupancy, uncertainty, operational risk, and profiling overhead distinct. Later sections introduce influence domains, task taxonomies, Task Behavior Graphs, feature mapping, and profiling-policy selection as one way to structure that problem.

References

[1] Yue Cheng, Ali Anwar, and Xuejing Duan. Analyzing alibaba's co-located datacenter workloads. In *2018 IEEE International Conference on Big Data*, Big Data '18. IEEE, 2018.

[2] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[3] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation*, OSDI '04, pages 137–150. USENIX Association, 2004.

[4] Christina Delimitrou and Christos Kozyrakis. Paragon: Qos-aware scheduling for heterogeneous datacenters. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '13. ACM, 2013.

[5] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and qos-aware cluster management. In *Proceedings of the Nineteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '14. ACM, 2014.

[6] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation*, NSDI '11. USENIX Association, 2011.

[7] OpenTelemetry. Opentelemetry documentation: Sampling. `https://opentelemetry.io/docs/concepts/sampling/`, 2025. Accessed 2026-05-03.

[8] OpenTelemetry. Opentelemetry documentation: Performance. `https://opentelemetry.io/docs/zero-code/java/agent/performance/`, 2026. Accessed 2026-05-03.

[9] OpenTelemetry. Opentelemetry specification: Metrics data model. `https://opentelemetry.io/docs/specs/otel/metrics/data-model/`, 2026. Accessed 2026-05-03.

[10] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the Third ACM Symposium on Cloud Computing*, SoCC '12. ACM, 2012.

[11] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical report, Google, 2010.

[12] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems*, EuroSys '15. ACM, 2015.

[13] Qi Zhang, Joseph L. Hellerstein, and Raouf Boutaba. Characterizing task usage shapes in google compute clusters. In *Proceedings of the 5th International Workshop on Large Scale Distributed Systems and Middleware*, LADIS '11. ACM, 2011.

A Extended Terminology

This appendix provides the complete terminology reference used by the paper. Section 2 intentionally keeps only the core terms needed for the main model. The extended glossary is organized by semantic area so that the main text can introduce concepts progressively while still keeping a precise reference for task entities, behavior classes, execution context, observations, influence domains, resource demand, worker occupancy, uncertainty, risk, graph structure, profiling policy, manifests, and queue-worker lifecycle stages.

A.1 Task Entities

Table 3: Task entity terms.

Term	Definition
task	A generic unit of work handled by a queue-worker system. The term is used only when the distinction between task kind, task instance, and task attempt is not important.
task kind	The semantic type of work submitted to a queue, such as copying a replication partition, rebuilding a cache entry, resizing an image, repairing a database record, dispatching a webhook, or validating a document. A task kind describes what work is requested, but it does not fully determine how that work behaves under every execution context.
task instance	A concrete logical unit of work placed into a queue. A task instance belongs to a task kind and may produce one or more execution attempts.
task attempt	One concrete execution attempt of a task instance. Retries, lease expiration, worker failure, cancellation, or rescheduling may cause a single task instance to have multiple attempts. Resource demand, worker occupancy, observations, outcomes, and residuals are usually measured or estimated at attempt granularity.
task family	A group of task kinds that share a similar behavior-graph structure or profiling model. A task family can be used when one graph template applies to several related task kinds.
task identity	The identifying metadata of a task, including task kind, task version, queue, producer, tenant, priority, and deadline. Task identity identifies the work but does not itself define the cost of executing it.
task version	The version of a task kind, handler contract, payload schema, or behavior manifest. Task versions are useful for compatibility between historical profiles, manifests, and schema versions.
queue	The queue through which a task instance is submitted, stored, reserved, and eventually consumed by a worker. A queue name may provide useful context, but it is not a complete behavior model.
producer	The component that creates and submits a task instance to a queue. The producer may provide metadata or declared hints, but it does not fully determine the execution context.

Term	Definition
worker	An executor that dequeues, reserves, executes, retries, acknowledges, discards, or fails task attempts. In this paper, a worker is treated as an execution boundary rather than merely a message consumer.
worker pool	A set of workers that execute task attempts. Worker-pool capacity and worker-pool occupancy are central to queue-worker profiling.
tenant	A user, customer, namespace, application, or logical group that consumes execution capacity. Tenants are relevant for isolation, fairness, consequence analysis, and operational risk.
tenant class	A category of tenants based on SLO, priority, plan, criticality, or operational risk. A tenant class is distinct from a task kind.
priority	A scheduling or ordering attribute assigned to a task instance. Priority may influence profiling or admission decisions, but it is not itself resource demand or operational risk.
deadline	A completion target or time constraint associated with a task instance. A deadline may affect consequence and risk, but it is distinct from observed latency.

A.2 Task Behavior

Table 4: Task behavior terms.

Term	Definition
task behavior	The observed or estimated execution pattern of a task kind, task instance, or task attempt under a particular execution context. Task behavior includes resource demand, worker occupancy, retry behavior, dependency wait, outcome, uncertainty, and risk-relevant effects.
behavior profile	A summarized description of how a task kind behaves under a class of execution contexts. A behavior profile may include demand distributions, occupancy distributions, dominant behavior classes, uncertainty, risk class, and relevant features.
behavior class	An analytical category derived from observed or estimated behavior. Examples include compute-dominant, dependency-bound, retry-amplifying, worker-slot-heavy, payload-sensitive, state-dependent, or high-tail-risk behavior.

Term	Definition
behavior mode	A context-dependent mode of behavior for a task kind. The term emphasizes that the same task kind may behave differently under different execution contexts.
compute-dominant behavior	Behavior in which CPU demand is the dominant active resource dimension. A task kind may exhibit compute-dominant behavior without being intrinsically a CPU task in all contexts.
memory-pressure-sensitive behavior	Behavior that is sensitive to memory pressure, allocation rate, working-set size, or garbage-collection effects. This behavior is not necessarily identical to being memory-bound.
disk-IO-dominant behavior	Behavior in which disk reads, disk writes, storage throughput, or I/O wait dominates execution. Disk I/O demand is distinct from disk pressure in the execution environment.
network-sensitive behavior	Behavior that changes substantially with network condition, such as packet loss, jitter, retransmissions, bandwidth changes, or timeouts.
dependency-bound behavior	Behavior constrained primarily by external dependencies, such as databases, APIs, remote services, rate limits, quotas, or connection pools.
retry-amplifying behavior	Behavior in which retries substantially increase work, worker occupancy, or operational risk. Retry count may be a symptom or outcome rather than an independent cause.
worker-slot-heavy behavior	Behavior with high worker-slot time relative to active resource usage. This class is important for queue workers because a task may consume little CPU but hold scarce worker capacity for a long time.
payload-sensitive behavior	Behavior that depends strongly on payload size, input shape, record count, object count, or batch size. Payload-sensitive behavior can still be uncertain if context or hidden state varies.
state-dependent behavior	Behavior that depends on domain-specific or internal system state, such as replica divergence, cache state, compaction debt, segment fragmentation, or dirty data volume.
high-tail-risk behavior	Behavior in which rare slow executions or high-percentile outcomes are operationally important. High-tail-risk behavior is distinct from high average cost.
stable behavior	Behavior with low residual error, low drift, and low variability under comparable execution contexts. Stable behavior may still be expensive.

Term	Definition
unstable behavior	Behavior with high variability, drift, residual error, multimodality, or context sensitivity. Unstable behavior is not necessarily high-cost in every attempt.

A.3 Execution Context

Table 5: Execution-context terms.

Term	Definition
execution context	The environment and state under which a task attempt executes. Execution context includes worker state, node pressure, dependency health, network condition, disk pressure, cache state, tenant pressure, retry state, and other runtime or domain-specific conditions.
context factor	A feature of the execution context that may influence task behavior. Context factors are modeled as influences, not automatically as proven causes.
worker state	The local state of the worker during a task attempt, including local runtime pressure, running work, memory state, garbage-collection pressure, and local scheduling delay.
worker pressure	Pressure on a worker or worker pool, such as busy worker slots, saturated local queues, high utilization, or local scheduling delay.
node pressure	Pressure on the host, virtual machine, container, or node on which the worker runs. Examples include CPU saturation, memory pressure, disk pressure, and network saturation.
dependency health	The operational condition of an external dependency, such as latency, timeout rate, error rate, saturation, quota pressure, or availability.
dependency capacity	The available capacity of an external dependency, such as connection-pool capacity, rate-limit budget, service quota, or downstream throughput.
network condition	The state of the network path relevant to a task attempt, including round-trip time, jitter, packet loss, retransmissions, bandwidth, timeouts, and availability.
network instability	A normalized feature representing unstable network behavior. It may be derived from packet loss, jitter, retransmissions, timeout rate, bandwidth variation, or recent network failures.

Term	Definition
replica state	The state of a replica, shard, peer, or replica set involved in a task attempt. Replica state is relevant to replication, repair, synchronization, and consistency tasks.
replica health	A normalized feature representing the operational condition of a replica or peer. Replica health is related to, but distinct from, replication lag.
replication lag	An observation or signal describing how far a replica is behind another source of truth. It may be measured in bytes, records, operations, versions, or time.
disk pressure	Pressure on the storage subsystem, such as queue depth, read latency, write latency, I/O wait, or saturation. Disk pressure is an execution-context factor, not the same as disk I/O demand.
cache state	The condition of a cache relevant to a task attempt, such as warm, cold, stale, evicted, partially populated, or inconsistent.
cache warmth	A normalized feature representing how prepared or populated a cache is for the task attempt. Cache warmth can influence resource demand and latency.
data locality	The proximity of required data to the worker or execution environment. Examples include local, remote, cross-zone, cross-region, or external storage access.
tenant pressure	Pressure or quota usage associated with a tenant, such as used concurrency, rate-limit consumption, or tenant-level backlog.
retry state	The state of the retry cycle for a task instance or attempt, including attempt number, previous errors, backoff state, retry budget, and retry history.
hidden system dynamics	Unobserved or partially observed system state that affects task behavior, such as internal dependency queues, lock contention, cache eviction history, data fragmentation, noisy neighbors, kernel behavior, or runtime effects. Hidden dynamics are represented through residual error, uncertainty, and reduced confidence when not captured by known features.

A.4 Observations, Signals, and Features

Table 6: Observation, signal, and feature terms.

Term	Definition
observation	Any measurement, event, trace, profile sample, counter, log entry, or domain-specific signal exposed by a queue, worker runtime, operating system, container runtime, dependency, tracing system, or domain component.
raw observation	An unnormalized measurement or event before interpretation. A raw observation does not by itself have stable model meaning.
metric	A time-series, counter, gauge, or histogram observation. Metrics are one kind of observation; traces, logs, profiles, and domain events are observations too.
counter	A metric that counts events, operations, bytes, retries, failures, or other monotonic quantities. A counter may represent a symptom or outcome rather than a root influence.
gauge	A metric representing a current value, such as queue depth, active workers, available connections, or current memory usage.
histogram	A metric representing a distribution of observed values, such as durations, latencies, sizes, or per-attempt costs.
trace	An execution trace composed of spans. A trace exposes execution structure and timing, but it does not by itself define task-kind-specific relevance, causality, feature mapping, or profiling policy.
span	A timed unit inside a trace representing an operation, processing stage, dependency call, or instrumented region. A span is not the same as a task attempt.
log	A structured or unstructured event record emitted by a system. Logs may expose status, error classes, domain events, or diagnostic information.
runtime profile	A CPU, heap, allocation, lock, blocking, or runtime-level profile. A runtime profile is distinct from a behavior profile.
signal	An interpreted, aggregated, or derived observation that indicates a condition relevant to the model. A signal is an intermediate layer between raw observation and canonical feature.
canonical feature	A typed and normalized model input derived from task metadata, execution context, raw observations, or signals. Task Behavior Graphs operate over canonical features rather than arbitrary raw metrics.

Term	Definition
semantic feature	An informal synonym for a feature with explicit semantic meaning. This paper uses the term canonical feature when referring to model inputs with defined type, range, unit, influence domain, and intended meaning.
feature mapping	The process of converting raw observations, signals, task metadata, and context into canonical features. Feature mapping is separate from graph evaluation.
normalization	The process of converting values into a defined type, unit, scale, bucket, or range. For example, network measurements may be normalized into an instability score in $[0, 1]$.
aggregation	The process of combining observations over a time window, group, task kind, attempt set, or distribution. Examples include sum, average, maximum, p95, and p99.
feature registry	A registry of canonical features, including names, types, ranges, units, influence domains, and intended meaning.
metric descriptor	A descriptor of a raw metric, including source, unit, label set, aggregation, cardinality, and mapping to canonical features.
feature descriptor	A descriptor of a canonical feature, including type, range, unit, semantic meaning, influence domain, and expected use in the model.
cardinality	The number of distinct time series or label combinations produced by a metric. Cardinality affects telemetry overhead and storage cost.
high-cardinality label	A metric label that can create a large or unbounded number of series, such as task id, user id, object id, URL, or unbounded tenant label.
observation window	The time window over which observations are aggregated or interpreted. For example, network instability may be computed over the last 30 seconds.
freshness	How recent and applicable an observation is to the current task attempt. Stale observations may reduce confidence.

A.5 Influence Domains

Table 7: Influence domain terms.

Term	Definition
influence domain	A semantic role assigned to a feature, observation, output, or model concept in the task behavior model. Influence domains are not metric domains: they group model elements by role rather than by telemetry source.
task identity domain	The influence domain containing identifiers and metadata that describe which task is being executed, such as task kind, version, queue, tenant, producer, priority, and deadline.
intrinsic demand domain	The influence domain describing work contained in the task itself, such as payload size, record count, object count, partition size, input format, operation mode, or batch size.
execution context domain	The influence domain describing the environment and state under which the task attempt executes, such as network condition, dependency health, replica state, disk pressure, cache warmth, worker pressure, and retry state.
resource demand domain	The influence domain describing active resource dimensions required or consumed by the task attempt, such as CPU time, memory peak, disk I/O, network I/O, runtime allocations, or accelerator usage. Waiting on dependencies is not active resource demand by itself, although it can contribute to worker occupancy and operational risk.
worker occupancy domain	The influence domain describing bounded execution capacity held by the task attempt, such as worker-slot time, lease time, connection hold time, lock hold time, semaphore hold time, or tenant concurrency time. Wait-related dimensions such as dependency wait time may contribute to this domain when a task keeps bounded worker capacity while waiting.
outcome domain	The influence domain describing how the task attempt completed, such as success, failure, timeout, cancellation, retryable error, non-retryable error, dead-lettering, or lease expiration.
uncertainty domain	The influence domain describing unreliability or unpredictability in the model estimate or behavior profile, such as residual error, drift, variance, tail ratio, multimodality, missing features, or low sample count.
risk domain	The influence domain describing the consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Examples include queue starvation, retry storm, SLO violation, replication lag, tenant impact, and cascading overload.

Term	Definition
profiling observation domain	The influence domain describing what is observed, where it is observed, at what detail level, and under which overhead budget.

A.6 Resource Demand, Occupancy, Cost, and Overhead

Table 8: Resource demand, occupancy, cost, and overhead terms.

Term	Definition
resource demand	The expected or required active resource need of a task attempt along a resource dimension. Resource demand is what the model estimates before or during execution.
resource usage	The observed actual use of a resource during execution. Resource usage is measured after or during execution and is distinct from estimated demand.
active resource usage	Actual use of active resources such as CPU, memory, disk, network, runtime allocation, or accelerator resources. Active resource usage is distinct from waiting and bounded-capacity occupancy.
CPU time	The amount of CPU execution time consumed by a task attempt. CPU time is not the same as wall-clock duration.
CPU pressure	Pressure on CPU capacity in the worker, container, node, or co-located environment. CPU pressure is an execution-context factor, not the same as task CPU demand.
memory peak	The maximum memory used by a task attempt during execution. Memory peak is a resource-usage or resource-demand dimension.
memory pressure	Pressure on available memory in the worker, container, node, or runtime environment. Memory pressure is an execution-context factor.
allocation behavior	The allocation volume, allocation rate, allocation pattern, or allocation-induced runtime effect of a task attempt. Allocation behavior may influence memory pressure, garbage collection, and CPU time.
disk I/O	Disk reads, writes, operations, bytes, or I/O wait associated with a task attempt. Disk I/O demand is distinct from disk pressure.
network I/O	Network bytes, packets, requests, or transfer volume associated with a task attempt. Network I/O is distinct from network instability.

Term	Definition
wait time	Time spent waiting rather than actively consuming a hardware or runtime resource. Wait time may be caused by dependencies, locks, network transfer, backoff, throttling, local scheduling delay, or other blocking conditions.
dependency wait time	Time spent waiting for an external dependency, such as a database, API, remote service, or storage system. Dependency wait time is not active resource demand by itself, but it can increase wall-clock duration and worker occupancy when bounded worker capacity remains held during the wait.
worker occupancy	Bounded execution capacity held by a task attempt over time. Worker occupancy is a first-class cost component in queue-worker systems.
worker-slot time	The time during which a task attempt holds a worker execution slot. This is the core worker-occupancy dimension.
lease time	The time during which a task lease or reservation is held. Lease time may differ from worker-slot time.
connection hold time	The time during which a task attempt holds a connection to a dependency, database, broker, or external service.
lock hold time	The time during which a task attempt holds a lock. Lock hold time can affect other tasks and may contribute to contention.
lock wait time	The time spent waiting to acquire a lock. Lock wait time is distinct from lock hold time.
semaphore hold time	The time during which a task attempt holds a semaphore, concurrency token, or bounded execution permit.
tenant concurrency time	The time during which a task attempt consumes a tenant-level concurrency slot. This dimension is relevant to fairness and isolation.
wall-clock duration	Elapsed time from attempt start to attempt finish. Wall-clock duration is often related to worker-slot time, but the two are not necessarily identical.
latency	Delay observed by a caller, queue, worker, dependency, or system boundary. Latency is an outcome or symptom, not resource demand by itself.
service time	The time a task attempt spends being served or executed, excluding queue wait time. The exact boundary must be defined for the queue-worker system being analyzed.
queue wait time	The time between enqueue and dequeue or execution start. Queue wait time is distinct from worker-slot time.
task cost	An umbrella term for resource demand, worker occupancy, and execution overhead associated with a task attempt. Operational risk is not part of task cost.

Term	Definition
execution overhead	Overhead inherent to executing a task attempt, such as serialization, input fetching, setup, acknowledgement, commit, cleanup, or retry bookkeeping.
profiling overhead	Overhead caused by profiling and instrumentation, including tracing, metrics, runtime profiling, high-cardinality labels, sampling decisions, and telemetry export.
telemetry overhead	CPU, memory, allocation, network, storage, or cardinality cost introduced by telemetry collection and export.

A.7 Uncertainty, Confidence, Residuals, and Risk

Table 9: Uncertainty, confidence, residual, and risk terms.

Term	Definition
uncertainty	Unreliability or unpredictability of a task behavior estimate. Uncertainty may come from missing features, hidden state, measurement error, drift, heavy tails, multimodality, or insufficient history.
entropy	A possible measure or family of measures for unpredictability. The term should only be used after specifying what distribution, signal, or behavior process is being measured.
confidence	The degree of trust in an estimate, behavior profile, graph edge, feature mapping, or model assumption. Confidence is distinct from uncertainty.
residual error	The part of observed behavior not explained by the current model estimate. Residual error exposes missing features, hidden influences, drift, or model mismatch.
prediction error	The difference between predicted and observed value for a resource, occupancy, duration, outcome, or risk-relevant dimension.
drift	A change in task behavior, feature mapping, graph relationship, context, or profile over time. Drift is a source of uncertainty and reduced confidence.
variance	The statistical spread of observed values. Variance is a variability signal but is not identical to entropy.
tail ratio	A ratio comparing a tail quantile with a central quantile, such as p95/p50 or p99/p50. Tail ratios help identify high-tail-risk behavior.
multimodality	The presence of multiple behavioral regimes or modes in the observed behavior of a task kind or task family.

Term	Definition
operational risk	The consequence-weighted danger of underestimating, misclassifying, or mishandling a task attempt. Operational risk is separate from task cost.
consequence	The severity of the effect caused by an incorrect estimate or control decision, such as delayed processing, queue starvation, SLO violation, replication lag, or cascading overload.
blast radius	The scope affected by an error or overload event, such as a single task, queue, worker pool, tenant, replica set, dependency, cluster, or control plane.
SLO violation risk	The operational risk that a task attempt, task kind, queue, tenant, or service will violate a service-level objective.
queue starvation risk	The risk that long-running or high-occupancy tasks delay or prevent other tasks from receiving execution capacity.
retry amplification risk	The risk that retries increase work, dependency pressure, queue occupancy, or worker occupancy enough to worsen overload.
cascade overload risk	The risk that local overload propagates to other queues, workers, dependencies, tenants, or control loops.

A.8 Task Behavior Graphs

Table 10: Task Behavior Graph terms.

Term	Definition
Task Behavior Graph	A typed graph associated with a task kind or task family. It connects canonical features across influence domains and supports estimation of resource demand, worker occupancy, uncertainty, and operational risk. A Task Behavior Graph is not assumed to prove causality by itself; it makes modeled influence assumptions explicit.
graph node	An element of a Task Behavior Graph representing a canonical feature, output, dimension, intermediate signal, or model concept. Each node belongs to one or more influence domains.
graph edge	A modeled influence relationship between two graph nodes. A graph edge is not a proven causal link unless validated by additional evidence.
influence role	The semantic role of a node or edge in the graph, such as baseline demand, contextual amplification, additive overhead, symptom, outcome link, risk propagation, uncertainty indicator, or profiling trigger.

Term	Definition
baseline demand edge	An edge that connects an intrinsic feature to a baseline resource-demand or occupancy estimate. For example, payload size may contribute to network I/O or disk I/O baseline demand.
contextual amplification edge	An edge by which an execution-context factor amplifies or suppresses demand, occupancy, uncertainty, or risk.
additive overhead edge	An edge that represents additional fixed or variable overhead, such as setup, serialization, acknowledgement, commit, or retry bookkeeping.
symptom edge	An edge that connects an influence to a symptom or observed effect, such as network degradation contributing to retries or longer wall-clock duration.
outcome link	An edge that connects modeled behavior to an outcome, such as timeout, failure, success, cancellation, or dead-lettering.
risk propagation edge	An edge that connects demand, occupancy, uncertainty, or outcome to an operational-risk dimension.
uncertainty indicator	A node or edge role indicating that a signal changes uncertainty or confidence rather than directly changing resource demand.
sensitivity	The estimated strength of a relationship between graph nodes. Sensitivity may be expert-defined, learned, calibrated, or represented as a qualitative level.
relevance	Whether a feature or relationship matters for a task kind, task family, or behavior profile. Relevance is distinct from sensitivity.
graph evaluation	The process of applying a Task Behavior Graph to canonical features and execution context in order to estimate demand, occupancy, uncertainty, residuals, and risk.
graph template	A reusable graph structure that can be specialized for multiple related task kinds within a task family.
model assumption	An explicit assumption encoded in the graph, feature mapping, or policy. Model assumptions should be validated, calibrated, or revised when observations contradict them.

A.9 Profiling and Observation Policy

Table 11: Profiling and observation-policy terms.

Term	Definition
profiling	The process of collecting, interpreting, and using observations to estimate task behavior. In this paper, profiling is broader than CPU profiling.
task profiling	Profiling logical task attempts in queue-worker systems in order to estimate resource demand, worker occupancy, uncertainty, and operational risk.
observation strategy	A decision about what to observe, where to observe it, how often to observe it, and under what overhead budget.
profiling policy	The selected observation and instrumentation strategy for a task kind, task attempt, or execution context.
accounting	Recording basic execution facts, such as start time, finish time, status, duration, input size, output size, resource usage, or occupancy.
minimal accounting	A low-overhead accounting level that records only essential task facts, such as start time, finish time, status, and basic size information.
detailed accounting	An accounting level that records more detailed resource, occupancy, stage, dependency, and outcome information.
tracing	Capturing execution structure and timing through traces and spans. Tracing is an observation method, not a behavior model by itself.
sampled tracing	Tracing only a subset of tasks, attempts, or traces in order to control overhead.
tail-aware profiling	A profiling strategy that preferentially observes slow, failed, high-risk, or tail-heavy attempts.
diagnostic profiling	Deep and usually expensive profiling used to investigate unexplained behavior, high residual error, drift, or model failure.
isolated execution profiling	Profiling in a more controlled or isolated execution environment in order to reduce interference or better attribute behavior.
cooperative instrumentation	Instrumentation in which worker or task code explicitly emits structured observations such as spans, timers, byte counts, record counts, dependency calls, stage boundaries, and domain-specific signals.
sampling	Selecting a subset of tasks, attempts, events, spans, traces, or profile samples for observation or retention.
head sampling	A trace-sampling strategy that decides whether to sample before the full trace is observed.
tail sampling	A trace-sampling strategy that decides whether to sample after seeing all or most of a trace.

Term	Definition
observation point	A point in the task lifecycle where observations can be collected, such as enqueue, dequeue, reserve, start, dependency call, retry, commit, acknowledge, or finish.
profiling budget	The allowed overhead for profiling and telemetry, including CPU, memory, allocation, storage, network, and cardinality costs.
hot path	The latency- or throughput-sensitive path of task execution where added instrumentation overhead can affect production behavior.

A.10 Manifests, Registries, and Domain Packs

Table 12: Manifest, registry, and domain-pack terms.

Term	Definition
Task Behavior Manifest	A declarative description of the behavior model for a task kind or task family. It may describe features, domains, graph nodes, graph edges, observation requirements, profiling policy defaults, known gaps, and versioning.
manifest	A machine-readable or human-readable specification used to declare part of the task behavior model.
domain pack	An extension package that provides domain-specific metrics, feature mappings, feature descriptors, graph templates, examples, or profiling defaults.
metric descriptor registry	A registry describing raw metrics and how they can be interpreted, aggregated, and mapped to canonical features.
graph declaration	The part of a manifest that declares graph nodes, graph edges, influence roles, sensitivities, confidence, and applicability conditions.
profiling policy declaration	The part of a manifest that declares default observation strategies, required observation points, sampling rules, and profiling overhead limits.
schema	A machine-validation structure for manifests, descriptors, registries, domain packs, or graph declarations.
versioning	The practice of assigning versions to schemas, manifests, domain packs, task kinds, feature registries, or graph structures in order to manage evolution and compatibility.
compatibility	The ability of manifests, schemas, domain packs, task versions, and graph structures to work together without semantic or validation conflicts.

Term	Definition
coverage	The set of task kinds, domains, features, metrics, observation points, or risks covered by a model, manifest, or domain pack. Coverage is not the same as global completeness.
known gap	A missing, unavailable, weak, or intentionally unsupported feature, observation, domain, or graph relationship. Known gaps should remain visible rather than being hidden inside the model.
residual visibility	The ability of the model to expose unexplained behavior through residual error, reduced confidence, or missing-feature indicators.
canonical namespace	A stable naming scheme for canonical features, resource dimensions, occupancy dimensions, and risk dimensions, such as `context.network.instability` or `occupancy.worker.slot_time`.

A.11 Queue-Worker Lifecycle Terms

Table 13: Queue-worker lifecycle terms.

Term	Definition
enqueue	The lifecycle point at which a task instance is submitted to a queue.
dequeue	The lifecycle point at which a worker takes a task instance from a queue for processing.
reserve	The lifecycle point at which a worker or worker framework claims a task instance, often through a lease, visibility timeout, or reservation mechanism.
start	The lifecycle point at which a task attempt begins execution.
input fetch	The stage in which the worker fetches external input, payload data, referenced objects, or domain state required by the task attempt.
deserialize	The stage in which the worker decodes, parses, or prepares the task payload or input data for processing.
processing stage	A logical phase inside task execution, such as validation, transformation, dependency call, write, commit, or cleanup.
dependency call	An interaction with an external service, API, database, storage system, broker, or remote dependency during task execution.

Term	Definition
retry	A repeated execution attempt or repeated operation after failure, timeout, lease expiration, cancellation, or retryable error.
backoff	A delay before retrying a task attempt or operation. Backoff can contribute to worker occupancy if execution capacity remains held.
output write	The stage in which task output, state changes, generated data, or side effects are written.
commit	The stage in which task effects are finalized, persisted, checkpointed, or made visible.
acknowledge (ack)	The lifecycle point at which a worker confirms task completion to the queue or broker.
discard	The lifecycle outcome in which a task instance or attempt is intentionally dropped or ignored.
dead-letter	The lifecycle outcome in which a task is moved to a dead-letter queue or equivalent failure storage.
finish	The lifecycle point at which a task attempt ends, whether by success, failure, timeout, cancellation, or another terminal outcome.
lease expiration	The lifecycle event in which a task reservation expires before successful acknowledgement or completion.
cancellation	The lifecycle event in which a task attempt is explicitly stopped before normal completion.

B Extended Task Taxonomy

C Manifest Schema

D Algorithm Details

E Case Study Details

F Evaluation Methodology Details

G Reproducibility Checklist